VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

VALLABHI GUPTA(1BF24CS328)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019 Feb-2026

to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by VALLABHI GUPTA (1BF24CS328) , who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.

C04	Conduct practical experiments to implement the functionalities of Operating system.

Lab Program 1

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.
a)FCFS
b) SJF (pre-emptive & Non-preemptive) CODE:

a)FCFS

```
#include <stdio.h>
#include <limits.h> #include
<stdbool.h>
struct Process {
int pid;     int
arrival;     int
burst;     int
remaining;     int
completion;     int
turnaround;     int
waiting;
}; int
main() {
int n;
   printf("Enter number of processes: ");
scanf("%d", &n);     struct Process p[n];
for (int i = 0; i < n; i++) {
       printf("Enter Arrival Time and Burst Time for Process %d: ", i+1);
p[i].pid = i+1;
       scanf("%d %d", &p[i].arrival, &p[i].burst);
       p[i].remaining = p[i].burst;
   }
   int current_time = 0, completed = 0;
float avg_turnaround = 0, avg_waiting = 0;
while (completed < n) {         int idx = -1;
       int min_remaining = INT_MAX;
       for (int i = 0; i < n; i++) {
          if (p[i].arrival <= current_time && p[i].remaining > 0) {
if (p[i].remaining < min_remaining) {
                min_remaining = p[i].remaining;
                idx = i;
             }
```

```
            }
        }
        if (idx == -1) {
            current_time++;
        } else {            p[idx].remaining--;            current_time++;
            if (p[idx].remaining == 0) {                p[idx].completion =
current_time;                p[idx].turnaround = p[idx].completion -
p[idx].arrival;                p[idx].waiting = p[idx].turnaround -
p[idx].burst;                avg_turnaround += p[idx].turnaround;
avg_waiting += p[idx].waiting;                completed++;
            }
        }
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].arrival, p[i].burst,
               p[i].completion, p[i].turnaround, p[i].waiting);
    }

    printf("\nAverage Turnaround Time = %.2f", avg_turnaround / n);
    printf("\nAverage Waiting Time = %.2f\n", avg_waiting / n);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS> cd 'c:\Users\Sachin Gupta\OneDrive\Desktop\OS\outp
ut'
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output> & .\'lab1(FCFS).exe'
1BF24CS328
Enter number of processes: 4
Enter Burst Time and Arrival Time for Process 1: 7 0
Enter Burst Time and Arrival Time for Process 2: 3 8
Enter Burst Time and Arrival Time for Process 3: 4 3
Enter Burst Time and Arrival Time for Process 4: 6 5

Process AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P3      3       4       11      8       4
P4      5       6       17      12      6
P2      8       3       20      12      9

Average Turnaround Time = 9.75
Average Waiting Time = 4.75
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output>
```

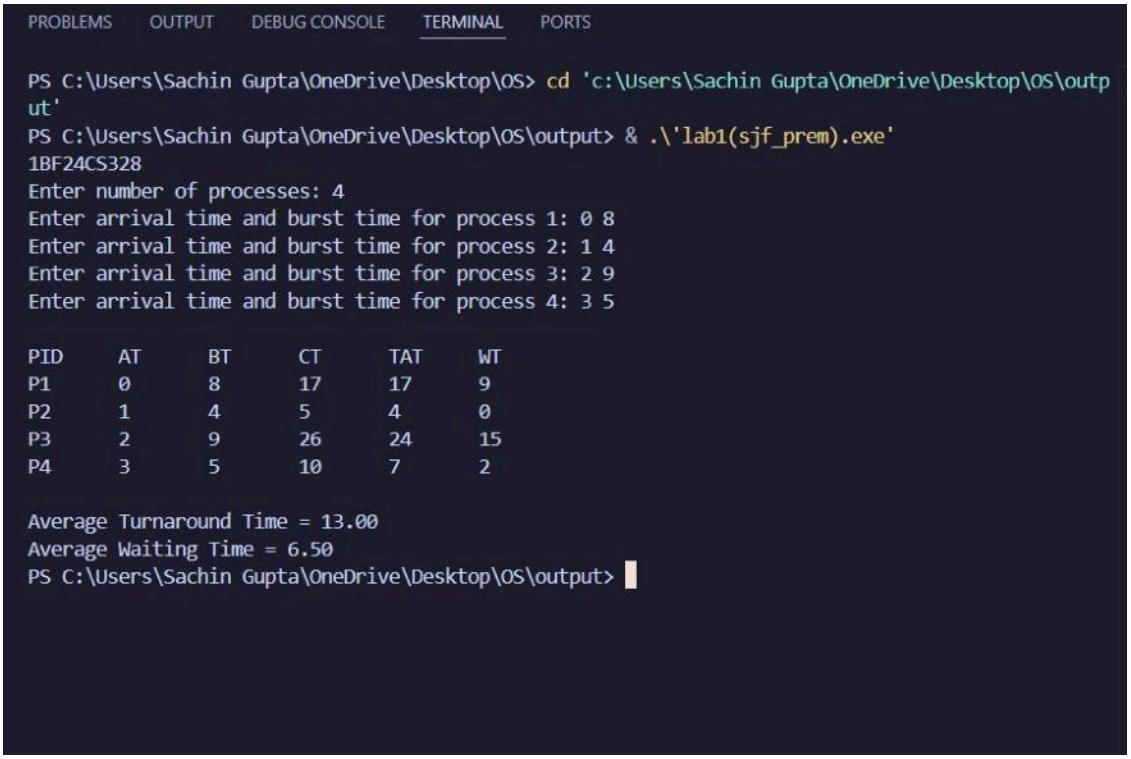
b) SJF (pre-emptive & Non-preemptive) SJF

pre-emptive:

```
#include <stdio.h> #include
<stdbool.h>
struct Process {
int pid;     int at;
int bt;     int rt;
int ct;     int tat;
int wt;     bool
finished;
}; int
main() {
int n;
   printf("Enter number of processes: ");
scanf("%d", &n);     struct Process p[n];
for (int i = 0; i < n; i++) {         p[i].pid
= i + 1;
       printf("Enter arrival time and burst time of P%d: ", i + 1);
scanf("%d %d", &p[i].at, &p[i].bt);         p[i].rt = p[i].bt;
p[i].finished = false;
   }
   int current_time = 0, completed = 0;
float total_tat = 0, total_wt = 0;
   while (completed < n) {
int idx = -1;
       int min_rt = 1e9;         for
(int i = 0; i < n; i++) {
          if (!p[i].finished && p[i].at <= current_time && p[i].rt < min_rt) {
min_rt = p[i].rt;                 idx = i;
          }         }         if
(idx == -1) {
current_time++;
       } else {             p[idx].rt--;
current_time++;             if (p[idx].rt == 0)
{                 p[idx].ct = current_time;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;
total_tat += p[idx].tat;                 total_wt
+= p[idx].wt;                 p[idx].finished =
true;
              completed++;
          }
       }
   }
   printf("\nSJF Preemptive Scheduling (SRTF):\n");
```

```
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);
return 0;
}
```

OUTPUT:

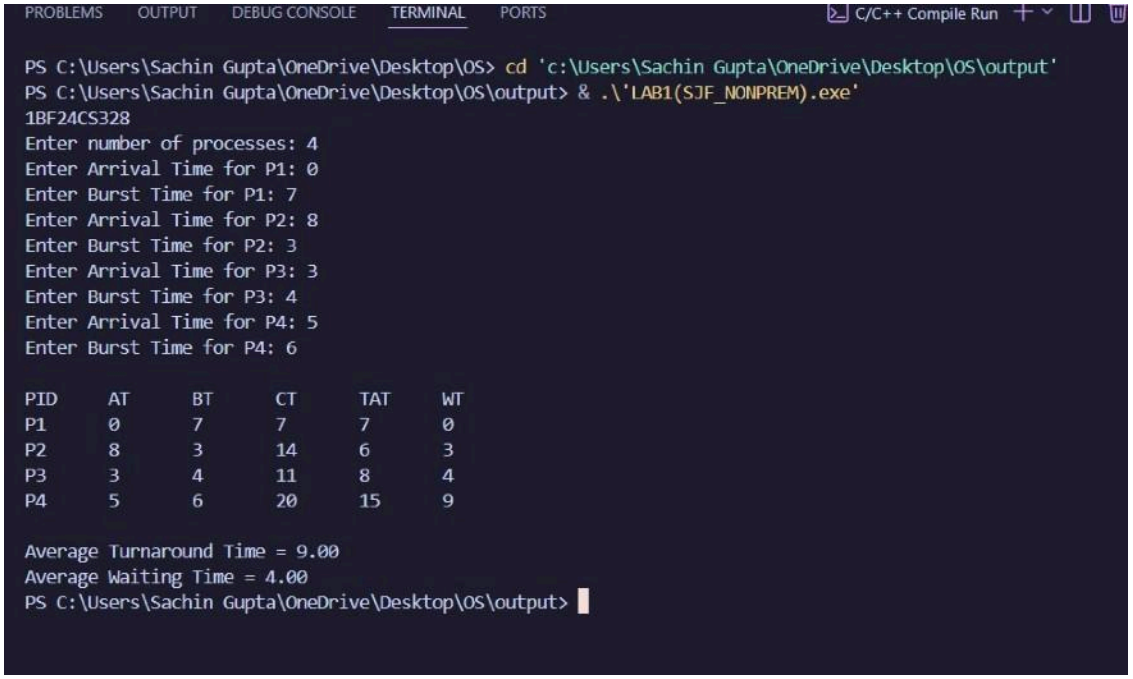


SJF non-preemptive:

```
#include <stdio.h> #include
<stdbool.h>
struct Process {
int pid;     int at;
int bt;     int ct;
int tat;     int wt;
bool finished;
}; int
main() {
int n;
   printf("Enter number of processes: ");
scanf("%d", &n);     struct Process p[n];
for (int i = 0; i < n; i++) {         p[i].pid
= i + 1;
       printf("Enter arrival time and burst time of P%d: ", i + 1);
scanf("%d %d", &p[i].at, &p[i].bt);
       p[i].finished = false;
   }
   int current_time = 0, completed = 0;
float total_tat = 0, total_wt = 0;
   while (completed < n) {
int idx = -1;         int min_bt =
1e9;         for (int i = 0; i < n;
i++) {
           if (!p[i].finished && p[i].at <= current_time && p[i].bt < min_bt) {
min_bt = p[i].bt;
               idx = i;
           }         }         if
(idx == -1) {
current_time++;
       } else {
           p[idx].ct = current_time + p[idx].bt;
           p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;
total_tat += p[idx].tat;             total_wt
+= p[idx].wt;             current_time =
p[idx].ct;             p[idx].finished = true;
           completed++;
       }
   }
   printf("\nSJF Non-Preemptive Scheduling:\n");
   printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
            p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);
return 0; }
```

OUTPUT:



Priority pre-emptive:

```
#include <stdio.h> #include
<stdbool.h>

struct Process {
int pid;     int at;
int bt;     int rt;
int pr;     int ct;
int tat;     int wt;
bool finished;
};

int main() {
int n;
   printf("Enter number of processes: ");
scanf("%d", &n);
```

```
   struct Process p[n];     for
(int i = 0; i < n; i++) {
p[i].pid = i + 1;
       printf("Enter arrival time, burst time, and priority of P%d: ", i + 1);
scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);         p[i].rt = p[i].bt;
p[i].finished = false;
   }

   int current_time = 0, completed = 0;
float total_tat = 0, total_wt = 0;

   while (completed < n) {
int idx = -1;
       int highest_priority = 1e9;
       for (int i = 0; i < n; i++) {
          if (!p[i].finished && p[i].at <= current_time && p[i].rt > 0) {
             if (p[i].pr < highest_priority) {
highest_priority = p[i].pr;
                 idx = i;
             }
          }
}

       if (idx == -1) {
current_time++;
       } else {
p[idx].rt--;
current_time++;

          if (p[idx].rt == 0) {
p[idx].ct = current_time;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;
total_tat += p[idx].tat;                 total_wt
+= p[idx].wt;                p[idx].finished =
true;
             completed++;
          }
       }
   }
```

```
    printf("\nPriority Scheduling (Preemptive):\n");
    printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].pr,
p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);     return 0; }
```

OUTPUT:

```
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS> cd 'c:\Users\Sachin Gupta\OneDrive\Desktop\OS\output'
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output> & .\'lab1(priority_preemtive).exe'
1BF24CS328
Enter number of processes: 5

Process 1
Enter Process ID, Arrival Time & Burst Time 1 0 3
Enter Priority (smaller number = higher priority): 3

Process 2
Enter Process ID, Arrival Time & Burst Time 2 1 4
Enter Priority (smaller number = higher priority): 2

Process 3
Enter Process ID, Arrival Time & Burst Time 3 2 6
Enter Priority (smaller number = higher priority): 4

Process 4
Enter Process ID, Arrival Time & Burst Time 4 3 4
Enter Priority (smaller number = higher priority): 6

Process 5
Enter Process ID, Arrival Time & Burst Time 5 5 2
Enter Priority (smaller number = higher priority): 10

Process AT      BT      Priority        WT      TAT
1       0       3       3               4       7
2       1       4       2               0       4
3       2       6       4               5       11
4       3       4       6               10      14
5       5       2       10              12      14

Ave
Ave   Focus folder in explorer (ctrl + click)
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output>
```

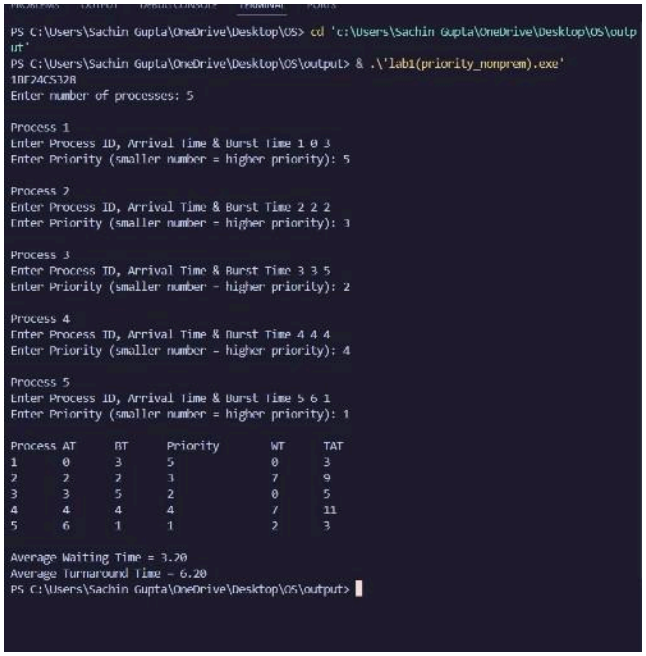
Priority non-preemptive:

```
#include <stdio.h> #include
<stdbool.h>
struct Process {
int pid;     int at;
int bt;     int pr;
int ct;     int tat;
```

```
int wt;     bool
finished;
}; int
main() {
int n;
   printf("Enter number of processes: ");
scanf("%d", &n);     struct Process p[n];
for (int i = 0; i < n; i++) {         p[i].pid
= i + 1;
       printf("Enter arrival time, burst time, and priority of P%d: ", i + 1);
scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
       p[i].finished = false;
   }
   int current_time = 0, completed = 0;
float total_tat = 0, total_wt = 0;
   while (completed < n) {
int idx = -1;         int
highest_priority = 1e9;
for (int i = 0; i < n; i++) {
          if (!p[i].finished && p[i].at <= current_time && p[i].pr <
highest_priority) {
             highest_priority = p[i].pr;
idx = i;
          }         }         if
(idx == -1) {
current_time++;
       } else {
          p[idx].ct = current_time + p[idx].bt;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;
total_tat += p[idx].tat;             total_wt +=
p[idx].wt;             current_time = p[idx].ct;
p[idx].finished = true;
          completed++;
       }
   }
   printf("\nPriority Scheduling (Non-Preemptive):\n");
   printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
       printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
            p[i].pid, p[i].at, p[i].bt, p[i].pr,
p[i].ct, p[i].tat, p[i].wt);
   }
      printf("\nAverage Turnaround Time: %.2f ms", total_tat /
n);      printf("\nAverage Waiting Time: %.2f ms\n", total_wt /
n);     return 0; }
```

OUTPUT:



Round Robin

```
#include <stdio.h>

int main()
{    int n,
tq;

   printf("Enter number of processes: ");
scanf("%d", &n);

   int pid[n], at[n], bt[n], rt[n];
int wt[n], tat[n], ct[n];

   for(int i = 0; i < n; i++)
   {        pid[i] = i + 1;

printf("\nProcess P%d\n", pid[i]);
```

```
        printf("Arrival Time: ");
scanf("%d", &at[i]);

        printf("Burst Time: ");
scanf("%d", &bt[i]);

        rt[i] = bt[i];
    }

    printf("\nEnter Time Quantum: ");
scanf("%d", &tq);

    int completed = 0;
int time = 0;

    while(completed < n)
    {
        int found = 0;

        for(int i = 0; i < n; i++)
        {
            if(at[i] <= time && rt[i] > 0)
            {
found = 1;

                if(rt[i] > tq)
{                       time += tq;
rt[i] -= tq;                    }
else                    {
time += rt[i];
ct[i] = time;
rt[i] = 0;
completed++;
                }
            }
        }

        if(found == 0)
time++;
    }
```

```
    float avgWT = 0, avgTAT = 0;

    for(int i = 0; i < n; i++)
    {         tat[i] = ct[i] -
at[i];          wt[i] = tat[i] -
bt[i];

        avgWT += wt[i];
avgTAT += tat[i];     }

      printf("\nPID\tAT\tBT\tCT\tTAT\tWT");

    for(int i = 0; i < n; i++)
    {
       printf("\nP%d\t%d\t%d\t%d\t%d\t%d",
             pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }

       printf("\nAverage Turnaround Time = %.2f", avgTAT / n);
printf("\nAverage Waiting Time    = %.2f", avgWT / n);

    return 0; }
```

OUTPUT:

```
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS> cd 'c:\Users\Sachin Gupta\OneDrive\Desktop\OS\output'
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output> & .\'lab1(roundrobin).exe'
USN:1BF24CS328
Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0 1 2 3 4
Enter burst times:
5 3 1 2 3

RRS scheduling:
PID     AT      BT      CT      TAT     WT      RT
1       0       5       13      13      8       0
2       1       3       12      11      8       1
3       2       1       5       3       2       2
4       3       2       9       6       4       4
5       4       3       14      10      7       5

Average turnaround time: 8.600000ms
Average waiting time: 5.800000ms
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output>
```

Lab Program 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the

processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#define MAX 10 typedef
struct {
   int pid, at, bt, rt, ct, tat, wt, q;
} P;
P p[MAX];
int n, tq = 2; int main() {     int time = 0,
completed = 0;     float total_tat = 0,
total_wt = 0;     printf("Enter number of
processes: ");     scanf("%d", &n);
for(int i = 0; i < n; i++) {
       p[i].pid = i + 1;
       printf("Enter AT BT Queue(1/2) for P%d: ", i+1);
       scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].q);
p[i].rt = p[i].bt;
   }
   while(completed < n) {
int executed = 0;        for(int i
= 0; i < n; i++) {
          if(p[i].q == 1 && p[i].rt > 0 && p[i].at <= time) {
executed = 1;               if(p[i].rt > tq) {                  time
+= tq;                  p[i].rt -= tq;
             } else {
time += p[i].rt;
p[i].rt = 0;
p[i].ct = time;
                completed++;
             }
          }
       }
       if(executed) continue;
for(int i = 0; i < n; i++) {
          if(p[i].q == 2 && p[i].rt > 0 && p[i].at <= time) {
executed = 1;               p[i].rt--;              time++;
if(p[i].rt == 0) {                  p[i].ct = time;
```

```
                completed++;
            }
break;
        }
      }
      if(!executed) time++;
   }
   for(int i = 0; i < n; i++) {
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;
total_tat += p[i].tat;        total_wt
+= p[i].wt;
   }
   printf("\nProcess\tQ\tAT\tBT\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++) {
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].q, p[i].at, p[i].bt,
p[i].ct, p[i].tat, p[i].wt);
   }
   printf("\nAverage TAT = %.2f", total_tat / n);     printf("\nAverage WT =
%.2f\n", total_wt / n);
   return 0; }
```

OUTPUT:

```
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS> cd 'c:\Users\Sachin Gupta\OneDrive\Desktop\OS\outp
ut'
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output> & .\'lab2(multilevelqueue).exe'
1BF24CS328
Enter number of processes: 4
Enter AT BT Queue(1/2) for P1: 0 4 1
Enter AT BT Queue(1/2) for P2: 0 3 1
Enter AT BT Queue(1/2) for P3: 0 8 2
Enter AT BT Queue(1/2) for P4: 10 5 1

Process Q       AT      BT      CT      TAT     WT
P1      1       0       4       6       6       2
P2      1       0       3       7       7       4
P3      2       0       8       20      20      12
P4      1       10      5       15      5       0

Average TAT = 9.50
Average WT = 4.50
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output>
```

Lab Program 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a)Rate- Monotonic

b)Earliest-deadline First

c)Proportional scheduling

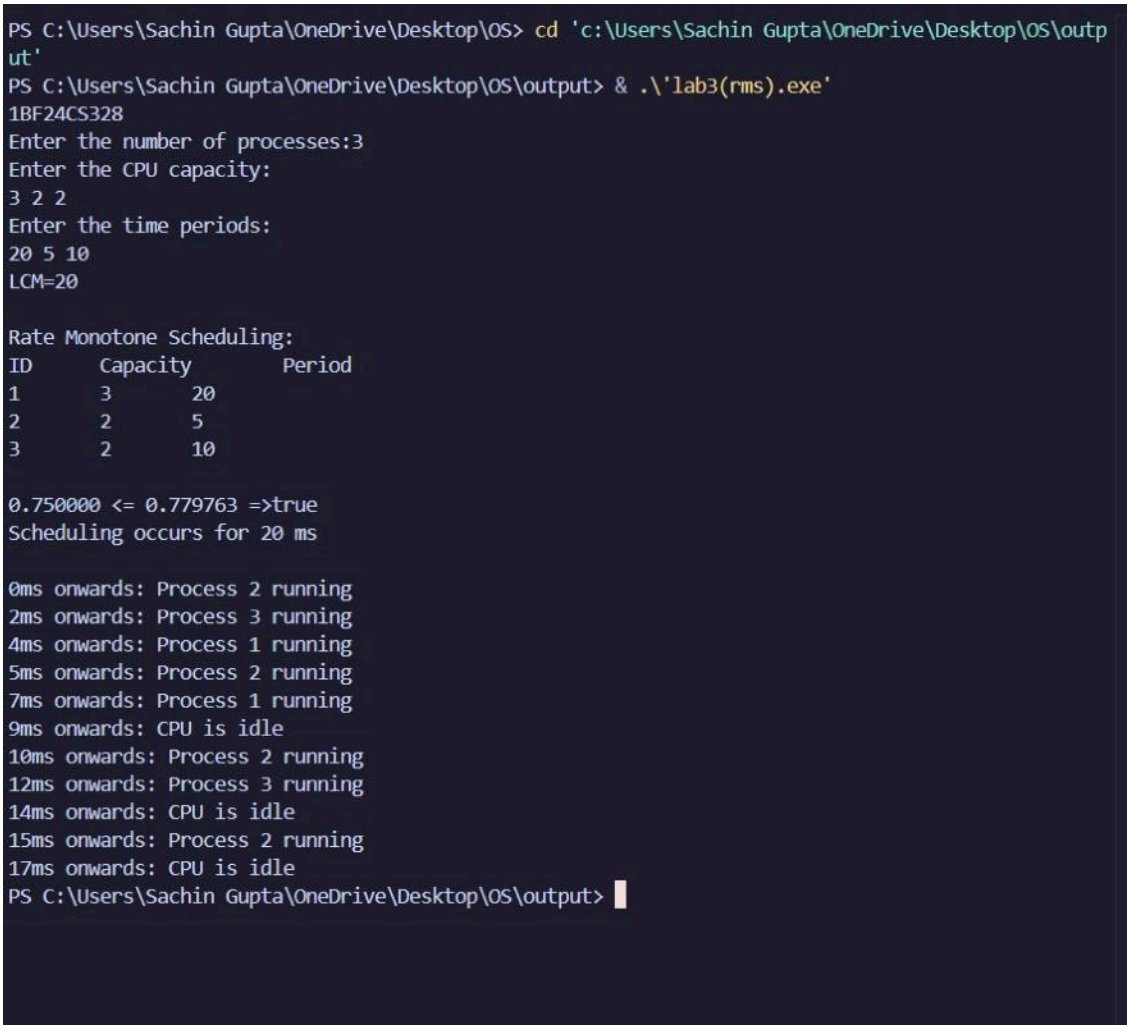
a)Rate- Monotonic

```
#include <stdio.h> #include
<math.h>
int gcd(int a, int b) {
while (b) {         a
%= b;         int temp
= a;         a = b;
b = temp;
    }
return a; }
int findLCM(int a, int b) {
return (a * b) / gcd(a, b);
} int main()
{
    printf("1BF24CS322\n");
    int n;
    printf("Enter the number of processes:");
scanf("%d", &n);
```

```
   int burst[n], period[n], remaining_burst[n], next_deadline[n];
printf("Enter the CPU burst times:\n");     for (int i = 0; i < n;
i++) scanf("%d", &burst[i]);     printf("Enter the time
periods:\n");     for (int i = 0; i < n; i++) scanf("%d",
&period[i]);     int lcm = period[0];     for (int i = 1; i < n; i++) {
       lcm = findLCM(lcm, period[i]);
   }
   printf("LCM=%d\n\n", lcm);
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");     for (int i =
0; i < n; i++) {
       printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
   }     printf("\n");
double utilization = 0;     for
(int i = 0; i < n; i++) {
       utilization += (double)burst[i] / period[i];
   }
   double bound = n * (pow(2.0, 1.0 / n) - 1);     printf("%f <= %f =>%s\n",
utilization, bound, (utilization <= bound) ? "true" :
"false");
   printf("Scheduling occurs for %d ms\n\n", lcm);
   for (int i = 0; i < n; i++) {
remaining_burst[i] = 0;
       next_deadline[i] = 0;
   }
   int current_proc = -1;     for (int t =
0; t < lcm; t++) {         for (int i = 0; i <
n; i++) {             if (t % period[i] == 0)
{                 remaining_burst[i] =
burst[i];
               next_deadline[i] = t + period[i];
           }
}
       int selected = -1;         for (int i
= 0; i < n; i++) {             if
(remaining_burst[i] > 0) {
               if (selected == -1 || period[i] < period[selected]) {
selected = i;
               }
           }
       }
       if (selected != current_proc) {
if (selected != -1) {
               printf("%dms onwards: Process %d running\n", t, selected + 1);
           } else {
```

```
            printf("%dms onwards: CPU is idle\n", t);
        }
        current_proc = selected;
    }       if (selected
!= -1) {
        remaining_burst[selected]--;
    }   }
return 0;
}
```

OUTPUT:

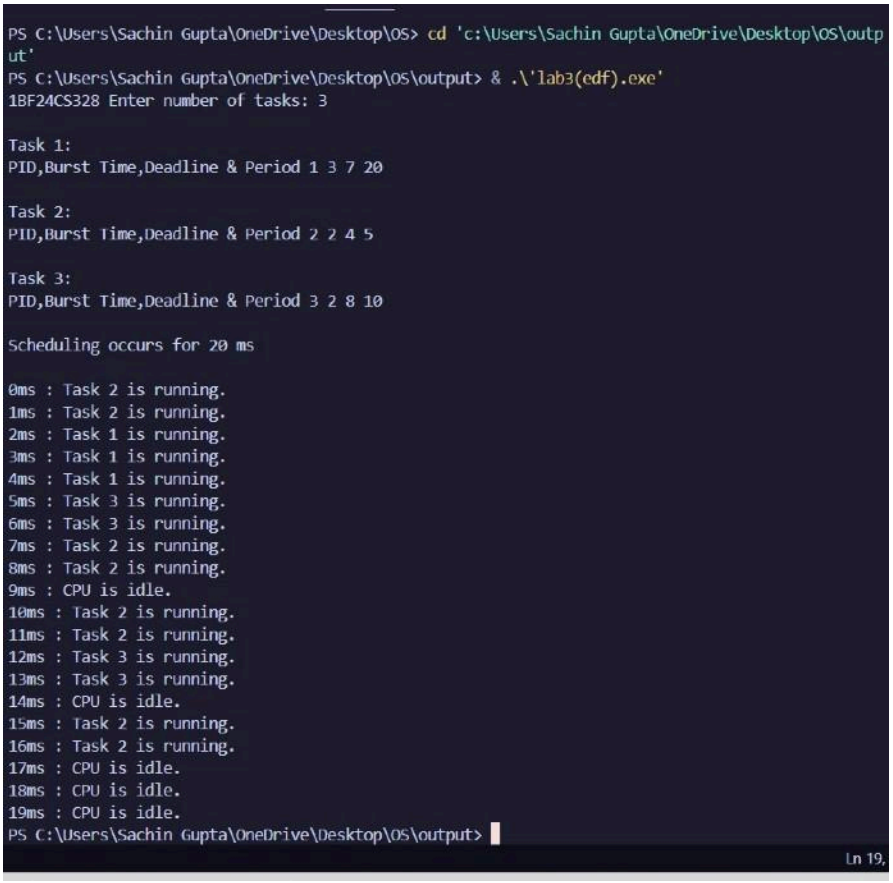


b)Earliest-deadline First

```
#include <stdio.h>
#define MAX 10
typedef struct {    int
pid;    int burst;    int
deadline_rel;    int
period;    int
remaining_burst;    int
current_deadline;
```

```
} Task; int main() {     prinG("1BF24CS322\n");     Task tasks[MAX];     int n,
total_Tme;     prinG("Enter number of processes: ");     if (scanf("%d", &n) != 1)
return 1;     for (int i = 0; i < n; i++) {         tasks[i].pid = i + 1;         prinG("Enter
burst, deadline, and period for Task %d: ", i + 1);         scanf("%d %d %d",
&tasks[i].burst, &tasks[i].deadline_rel, &tasks[i].period);
tasks[i].remaining_burst = 0;         tasks[i].current_deadline = 0;
   }
   prinG("Enter total scheduling Tme (e.g., 20): ");
scanf("%d", &total_Tme);
prinG("\nPID\tBurst\tDeadline\tPeriod\n");
   for (int i = 0; i < n; i++) {         prinG("%d\t%d\t%d\t\t%d\n", tasks[i].pid,
tasks[i].burst, tasks[i].deadline_rel, tasks[i].period);
   }
   prinG("\n--- Scheduling Log ---\n");     for (int t = 0; t <
total_Tme; t++) {             for (int i = 0; i < n; i++) {             if (t
% tasks[i].period == 0) {                 tasks[i].remaining_burst =
tasks[i].burst;                 tasks[i].current_deadline = t +
tasks[i].deadline_rel;
          }
      }
      int selected = -1;         int min_deadline = 9999;
for (int i = 0; i < n; i++) {             if
(tasks[i].remaining_burst > 0) {                 if
(tasks[i].current_deadline < min_deadline) {
min_deadline = tasks[i].current_deadline;
selected = i;
             }
          }
      }
      if (selected == -1) {
prinG("%dms : CPU is idle.\n", t);
      } else {
          prinG("%dms : Task %d is running.\n", t, tasks[selected].pid);
tasks[selected].remaining_burst--;
      }
   }
   return 0;
}
```

OUTPUT:

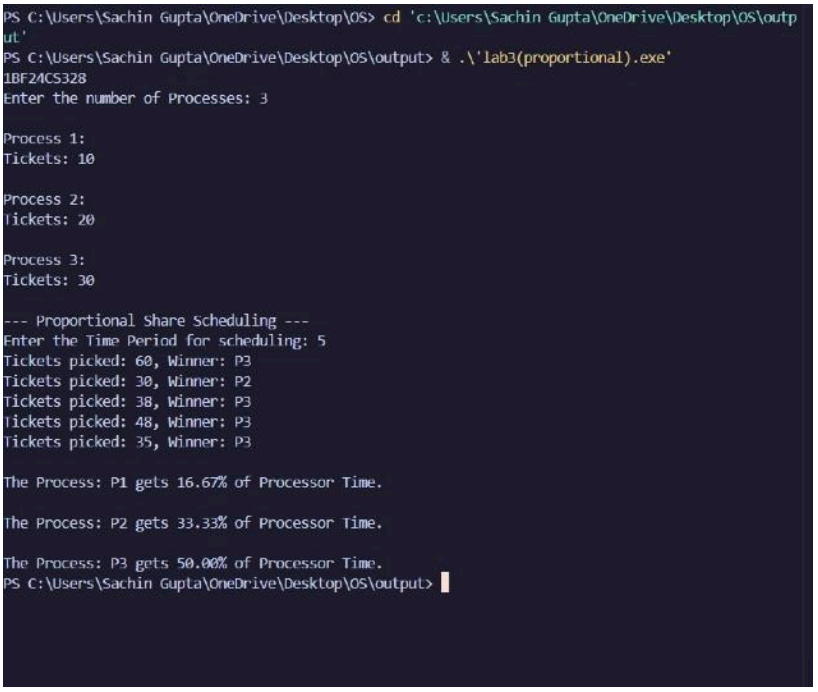


c)Proportional scheduling

```
#include <stdio.h>
#include <stdlib.h> #include
<time.h>

typedef struct {
char name[10];
int tickets; }
Process;

int main() {     int n,
total_tickets = 0;     float
total_T = 0.0;

   printf("Enter the number of Processes: ");
scanf("%d", &n);

   Process p[n];
srand(time(NULL));
```

```
   for (int i = 0; i < n; i++) {
printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");        scanf("%d",
&p[i].tickets);        total_tickets +=
p[i].tickets;        total_T +=
p[i].tickets;
   }

   printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;     scanf("%d", &m);

   for (int i = 0; i < m; i++) {
      int winning_ticket = rand() % total_tickets + 1;        int
accumulated_tickets = 0;        int winner_index = -1;

      for (int j = 0; j < n; j++) {
accumulated_tickets += p[j].tickets;            if
(winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
         }
      }
      printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
   }

   for (int i = 0; i < n; i++) {
      printf("\nThe Process: %s gets %.2f%% of Processor Time.\n",
p[i].name, ((float)p[i].tickets / total_T) * 100);
   }

   return 0; }
```

OUTPUT:



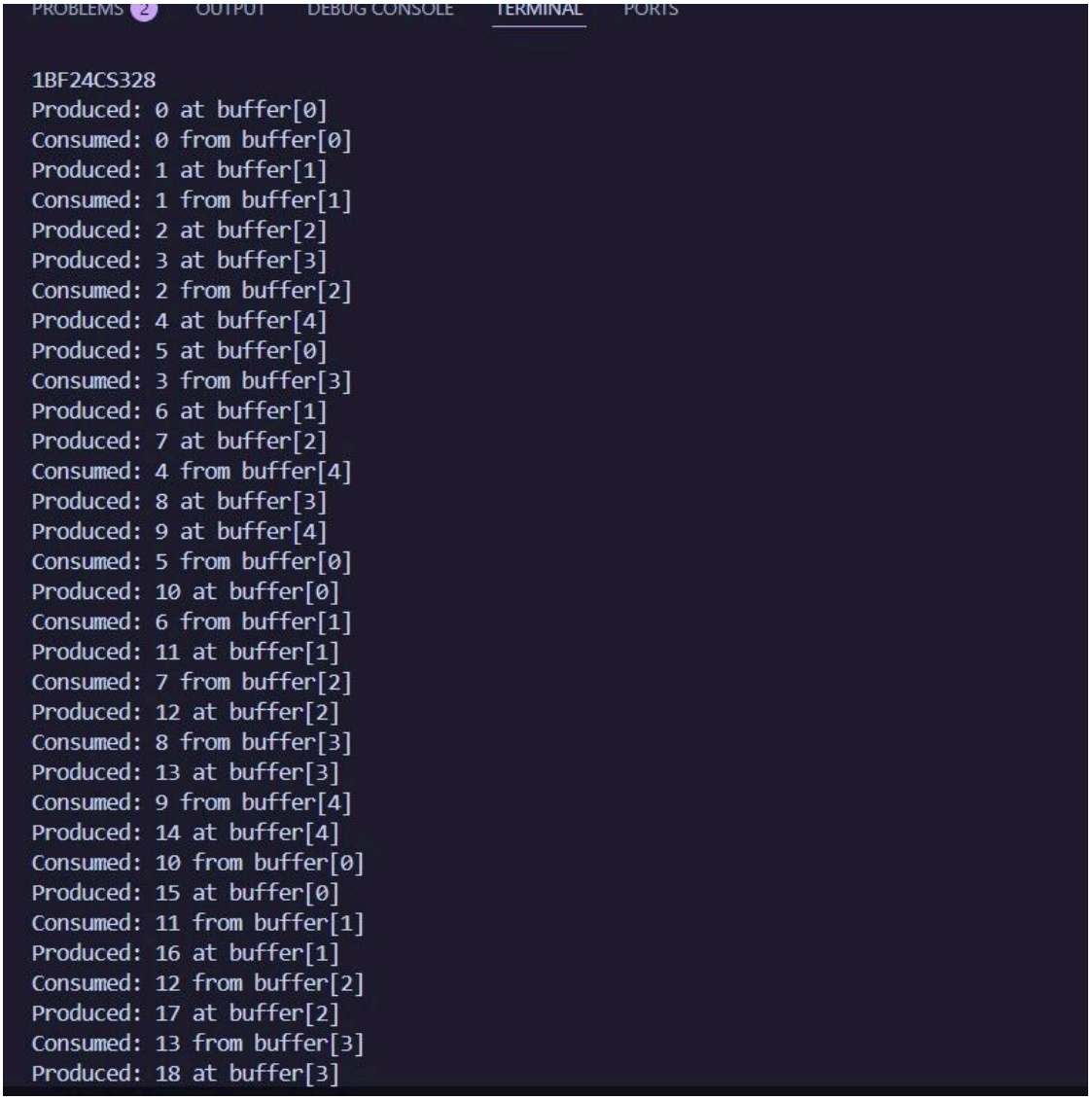
Lab Program 4 aWrite a C program to simulate producer-consumer problem using semaphores

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h> #define

BUFFER_SIZE 5

int buffer[BUFFER_SIZE int
in = 0, out = 0;

sem_t empty; sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
int item, i = 0;     while (1) {
item = i++;

      sem_wait(&empty);
pthread_mutex_lock(&mutex);
```

```
      buffer[in] = item;        prinG("Produced: %d at
buffer[%d]\n", item, in);        in = (in + 1) %
BUFFER_SIZE;

      pthread_mutex_unlock(&mutex);
sem_post(&full);

      sleep(1);
   }
   return NULL;
}

void* consumer(void* arg) {
   int item;     while (1) {
sem_wait(&full);
pthread_mutex_lock(&mutex);

      item = buffer[out];        prinG("Consumed: %d from
buffer[%d]\n", item, out);        out = (out + 1) %
BUFFER_SIZE;

      pthread_mutex_unlock(&mutex);
sem_post(&empty);

      sleep(2);
   }
   return NULL;
}

int main() {     pthread_t
prod, cons;

   sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);

   pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);
```

```
)
   pthread_join(prod, NULL);
pthread_join(cons, NULL);

   sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

   return 0; }
```

OUTPUT:



Lab Program 4

bWrite a C program to simulate the concept of Dining Philosophers problem.

```
#include <stdio.h>
```

```
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork pthread_t
philosophers[N];

void* philosopher(void* num) {
int id = *(int*)num;
   int left = id;
int right = (id + 1) % N;

   while (1) {
      printf("Philosopher %d is thinking.\n", id);
sleep(1);

      if (id % 2 == 0) {

         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d.\n", id, left);

         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);
} else {

         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);

         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d.\n", id, left);
      }

      printf("Philosopher %d is eating.\n", id);
sleep(2);

      pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
      printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
   }
```

```
    return NULL;
}

int main() {
    int i;     int
ids[N];

    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
    }

    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

OUTPUT:

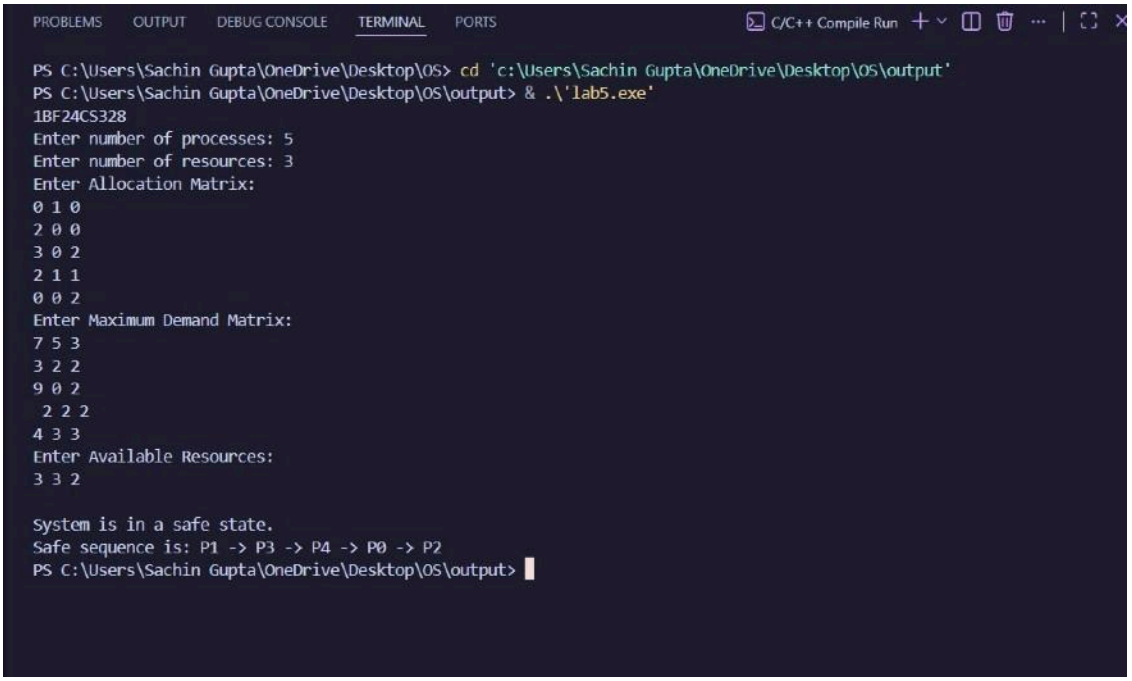
```
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS> cd 'c:\Users\Sachin Gupta\OneDrive\Desktop\OS\output'
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output> & .\'lab4(dining_threads).exe'
1BF24CS328
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
```

Lab Program 5 aWrite a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h> #include
<stdbool.h>

int main() {
   int n, m;
   printf("Enter the number of processes: ");
scanf("%d", &n);
   printf("Enter the number of resources: ");
scanf("%d", &m);

   int alloc[n][m], max[n][m], avail[m];
int need[n][m];

   printf("\nEnter the Allocation Matrix:\n");
   for (int i = 0; i < n; i++) {
for (int j = 0; j < m; j++) {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("\nEnter the Maximum Matrix:\n");
   for (int i = 0; i < n; i++) {
for (int j = 0; j < m; j++) {
         scanf("%d", &max[i][j]);
      }
   }

   printf("\nEnter the Available Resources:\n");
   for (int j = 0; j < m; j++) {
      scanf("%d", &avail[j]);
   }

   // Calculate Need Matrix
for (int i = 0; i < n; i++) {
for (int j = 0; j < m; j++) {
         need[i][j] = max[i][j] - alloc[i][j];
      }
}

   bool finish[n];
   for (int i = 0; i < n; i++) finish[i] = false;
```

```
   int safeSeq[n];
int work[m];
   for (int j = 0; j < m; j++) work[j] = avail[j];


   int count = 0;     while (count < n)
{        bool found = false;         for
(int i = 0; i < n; i++) {            if
(!finish[i]) {                bool possible
= true;               for (int j = 0; j < m;
j++) {                  if (need[i][j] >
work[j]) {                      possible =
false;                     break;
                }               }               if
(possible) {                  for (int j = 0;
j < m; j++) {
                    work[j] += alloc[i][j];
                }
                safeSeq[count++] = i;
finish[i] = true;                   found
= true;
             }
          }        }
if (!found) {
          printf("\nSystem is not in a safe state.\n");
          return 0;
       }
   }
   printf("\nSystem is in a safe state.\nSafe sequence is: ");
   for (int i = 0; i < n; i++) {
      printf("P%d ", safeSeq[i]);
   }
printf("\n");


   return 0; }
```

OUTPUT:



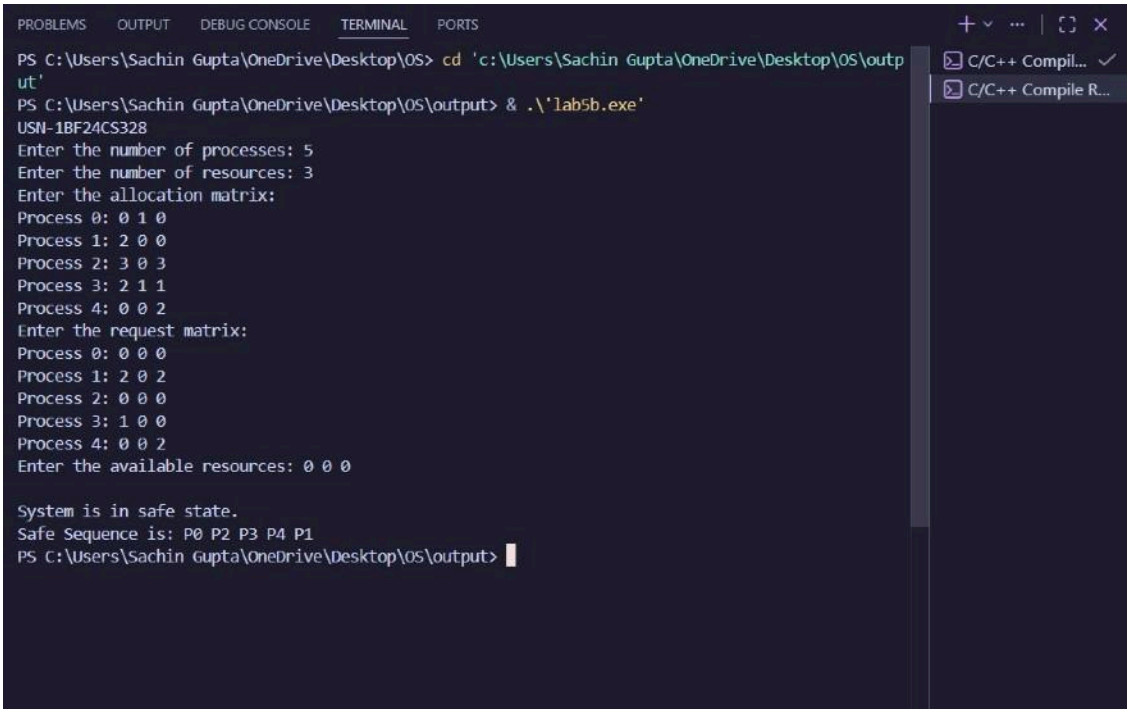
B.Write a C program to simulate Deadlock Detec4on

```
#include <stdio.h>

int main() {
   int n, m, i, j, k;

   prinG("Enter number of processes: ");
scanf("%d", &n);

   prinG("Enter number of resource types: ");
scanf("%d", &m);

   int AllocaTon[n][m], Request[n][m];
int Available[m];    int Finish[n];

   prinG("\nEnter AllocaTon Matrix:\n");
for(i = 0; i < n; i++)        for(j = 0; j < m;
j++)          scanf("%d", &AllocaTon[i][j]);

   prinG("\nEnter Request Matrix:\n");
for(i = 0; i < n; i++)        for(j = 0; j < m;
j++)          scanf("%d",
&Request[i][j]);
```

```
   prinG("\nEnter Available Resources:\n");
for(i = 0; i < m; i++)        scanf("%d",
&Available[i]);

   for(i = 0; i < n; i++)
Finish[i] = 0;
   int found;

   do {
found = 0;

      for(i = 0; i < n; i++) {
if(Finish[i] == 0) {
int canProceed = 1;

            for(j = 0; j < m; j++) {
if(Request[i][j] > Available[j]) {
canProceed = 0;                    break;
               }
            }

            if(canProceed) {
for(k = 0; k < m; k++)
                  Available[k] += AllocaTon[i][k];

               Finish[i] = 1;
found = 1;
            }
         }
      }
   } while(found);

int deadlock = 0;
```

```
   prinG("\nProcesses in Deadlock:\n");
for(i = 0; i < n; i++) {        if(Finish[i] ==
0) {            prinG("P%d ", i);
          deadlock = 1;
       }
   }

   if(deadlock == 0)        prinG("No
Deadlock Detected");     prinG("\n");

   return 0;
}
```

OUTPUT:



Lab Program 6:

Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit

b) Best-fit

c) First-fit #include <stdio.h>

```
void firstFit(int blockSize[], int m, int processSize[], int n)
{
   int allocaTon[n];

   for(int i = 0; i < n; i++)
allocaTon[i] = -1;

   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < m; j++)
      {
         if(blockSize[j] >= processSize[i])
         {
            allocaTon[i] = j;
            blockSize[j] -= processSize[i];
break;
         }
      }
   }
   prinG("\n--- First Fit ---\n");
   prinG("Process No.\tProcess Size\tBlock No.\n");

   for(int i = 0; i < n; i++)
   {
      prinG("%d\t\t%d\t\t", i + 1, processSize[i]);
```

```
        if(allocaTon[i] != -1)
printG("%d\n", allocaTon[i] + 1);
else
            printG("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocaTon[n];

    for(int i = 0; i < n; i++)
allocaTon[i] = -1;

    for(int i = 0; i < n; i++)
    {
        int bestIdx = -1;
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
bestIdx = j;
            }
        }

        if(bestIdx != -1)
        {
```

```
            allocaTon[i] = bestIdx;
blockSize[bestIdx] -= processSize[i];
        }
    }

    prinG("\n--- Best Fit ---\n");
    prinG("Process No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    {
        prinG("%d\t\t%d\t\t", i + 1, processSize[i]);

        if(allocaTon[i] != -1)
prinG("%d\n", allocaTon[i] + 1);
else
            prinG("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocaTon[n];

    for(int i = 0; i < n; i++)
allocaTon[i] = -1;

    for(int i = 0; i < n; i++)
    {
```

```
      int worstIdx = -1;

      for(int j = 0; j < m; j++)
      {
         if(blockSize[j] >= processSize[i])
         {
            if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
worstIdx = j;
         }
      }

      if(worstIdx != -1)
      {
         allocaTon[i] = worstIdx;
blockSize[worstIdx] -= processSize[i];
      }
   }

   prinG("\n--- Worst Fit ---\n");     prinG("Process
No.\tProcess Size\tBlock No.\n");

   for(int i = 0; i < n; i++)
   {
      prinG("%d\t\t%d\t\t", i + 1, processSize[i]);

      if(allocaTon[i] != -1)
prinG("%d\n", allocaTon[i] + 1);
else
```

```
            prinG("Not Allocated\n");
    }
}

int main()
{    int m,
n;

    prinG("Enter number of memory blocks: ");
scanf("%d", &m);
    int blockSize1[m], blockSize2[m], blockSize3[m];

    prinG("Enter sizes of %d memory blocks:\n", m);

    for(int i = 0; i < m; i++)
    {
        scanf("%d", &blockSize1[i]);
blockSize2[i] = blockSize1[i];
blockSize3[i] = blockSize1[i];
    }

    prinG("Enter number of processes: ");
scanf("%d", &n);

    int processSize[n];

    prinG("Enter sizes of %d processes:\n", n);
```

```
    for(int i = 0; i < n; i++)
scanf("%d", &processSize[i]);

    firstFit(blockSize1, m, processSize, n);
bestFit(blockSize2, m, processSize, n);
worstFit(blockSize3, m, processSize, n);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS> cd 'c:\Users\Sachin Gupta\OneDrive\Desktop\OS\outp
ut'
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output> & .\'lab6(mmu).exe'
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212
417 112 426

--- First Fit ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- Best Fit ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output>
```

LAB PROGRAM 7:

Write a C program to simulate page replacement algorithms. a) FIFO
b) LRU
c) Optimal

```
#include <stdio.h> #define

MAX 100

int isPresent(int frames[], int num_frames, int page)
{
    for (int i = 0; i < num_frames; i++)
    {
        if (frames[i] == page)
            return 1;
    }
    return 0;
}

void printFrames(int frames[], int num_frames)
{
printf("[");
    for (int i = 0; i < num_frames; i++)
    {
        if (frames[i] != -1)
            printf("%d ", frames[i]);
    }
    printf("]\n");
}

void FIFO(int pages[], int n, int num_frames)
{
    int frames[MAX];
int page_faults = 0;
int next = 0;

    for (int i = 0; i < num_frames; i++)
frames[i] = -1;     printf("\n--- FIFO Page
Replacement ---\n");     for (int i = 0; i < n; i++)
    {
        int page = pages[i];

        if (!isPresent(frames, num_frames, page))
        {
            frames[next] = page;             next
= (next + 1) % num_frames;
            page_faults++;
        }

        printf("Page %d -> ", page);
```

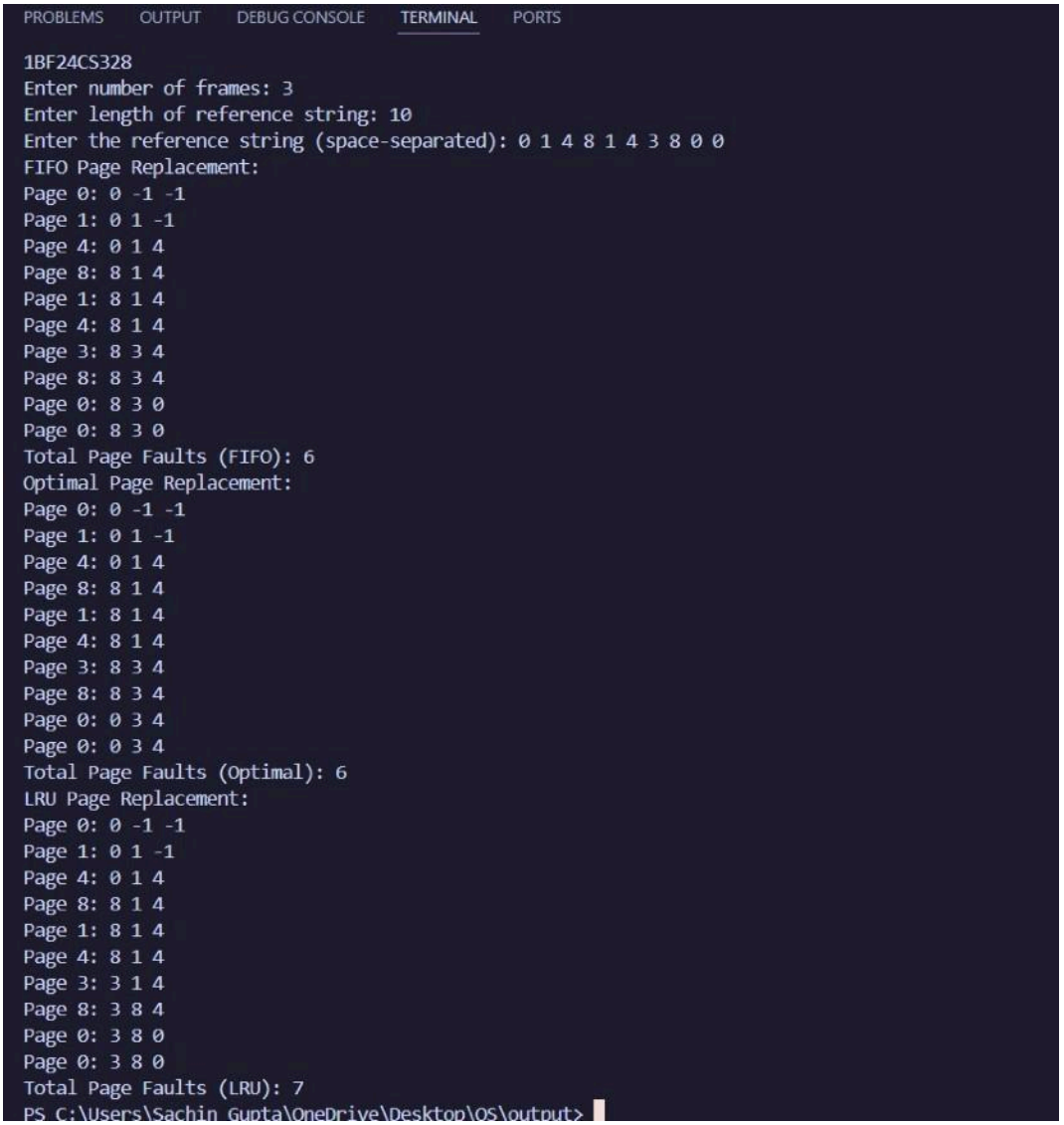
```
        printFrames(frames, num_frames);
    }

    printf("Total Page Faults (FIFO): %d\n", page_faults);
}

void LRU(int pages[], int n, int num_frames)
{
    int frames[MAX];
int lastUsed[MAX];
int page_faults = 0;

    for (int i = 0; i < num_frames; i++)
    {        frames[i]
= -1;
        lastUsed[i] = -1;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        int page = pages[i];
int found = 0;

        for (int j = 0; j < num_frames; j++)
        {
            if (frames[j] == page)
            {
found = 1;
lastUsed[j] = i;
break;
            }
}

        if (!found)
{            int pos =
-1;

            for (int j = 0; j < num_frames; j++)
            {
                if (frames[j] == -1)
                {
pos = j;
break;
                }
}
```

```
            if (pos == -1)
            {               int
lruIndex = 0;

                for (int j = 1; j < num_frames; j++)
                {
                    if (lastUsed[j] < lastUsed[lruIndex])
                        lruIndex = j;
                }

                pos = lruIndex;
            }

            frames[pos] = page;
lastUsed[pos] = i;
            page_faults++;
        }

        printf("Page %d -> ", page);
        printFrames(frames, num_frames);
    }

    printf("Total Page Faults (LRU): %d\n", page_faults);
}

void Optimal(int pages[], int n, int num_frames)
{
    int frames[MAX];
int page_faults = 0;
// Initialize frames
for (int i = 0; i <
num_frames; i++)
frames[i] = -1;
printf("\n--- Optimal
Page Replacement
---\n");

    for (int i = 0; i < n; i++)
    {
        int page = pages[i];

        if (!isPresent(frames, num_frames, page))
        {           int
pos = -1;

            for (int j = 0; j < num_frames; j++)
            {
```

```
            if (frames[j] == -1)
            {
pos = j;
break;
            }
}

          if (pos == -1)
          {               int farthest
= i;              int replaceIndex
= -1;

            for (int j = 0; j < num_frames; j++)
            {
int k;

                for (k = i + 1; k < n; k++)
                {
                    if (frames[j] == pages[k])
                        break;
                }

                if (k == n)
{
                    replaceIndex = j;
break;
                }
                if (k > farthest)
                {
farthest = k;
                    replaceIndex = j;
                }
            }

            pos = replaceIndex;
          }

          frames[pos] = page;
          page_faults++;
       }

       printf("Page %d -> ", page);
       printFrames(frames, num_frames);
   }

   printf("Total Page Faults (Optimal): %d\n", page_faults);
}
```

```
int main()
{
    int pages[MAX], n, num_frames;

    printf("Enter number of pages: ");
scanf("%d", &n);

    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
scanf("%d", &num_frames);

    FIFO(pages, n, num_frames);
    LRU(pages, n, num_frames);
Optimal(pages, n, num_frames);

    return 0;
}
```

OUTPUT


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
1BF24CS328
Enter number of frames: 3
Enter length of reference string: 10
Enter the reference string (space-separated): 0 1 4 8 1 4 3 8 0 0
FIFO Page Replacement:
Page 0: 0 -1 -1
Page 1: 0 1 -1
Page 4: 0 1 4
Page 8: 8 1 4
Page 1: 8 1 4
Page 4: 8 1 4
Page 3: 8 3 4
Page 8: 8 3 4
Page 0: 8 3 0
Page 0: 8 3 0
Total Page Faults (FIFO): 6
Optimal Page Replacement:
Page 0: 0 -1 -1
Page 1: 0 1 -1
Page 4: 0 1 4
Page 8: 8 1 4
Page 1: 8 1 4
Page 4: 8 1 4
Page 3: 8 3 4
Page 8: 8 3 4
Page 0: 0 3 4
Page 0: 0 3 4
Total Page Faults (Optimal): 6
LRU Page Replacement:
Page 0: 0 -1 -1
Page 1: 0 1 -1
Page 4: 0 1 4
Page 8: 8 1 4
Page 1: 8 1 4
Page 4: 8 1 4
Page 3: 3 1 4
Page 8: 3 8 4
Page 0: 3 8 0
Page 0: 3 8 0
Total Page Faults (LRU): 7
PS C:\Users\Sachin Gupta\OneDrive\Desktop\OS\output>